Formal Verification of First-Class Functions in Move

Wolfgang Grieskamp* and Teng Zhang and Vineeth Kashyap and Jake Silverman
Aptos Labs, Palo Alto, USA
*Corresponding author: wg@aptoslabs.com

***Abstract*—The Move Prover (MVP) is a formal verifier for smart contracts written in the Move programming language. Recently, Move on Aptos was extended with *higher-order functions*: function values that can be passed around, stored in data structs, and kept in persistent storage, enabling *dynamic dispatch*. This paper describes the novel representation of function values in the Move specification language and their implementation in MVP. We introduce *behavioral predicates* which characterize Move functions (aborts and pre/post conditions) by single-state or two-state predicates. We also introduce *state labels* for naming intermediate memory states in which expressions are evaluated and which allow to compose behavioral predicates to describe sequences of state transitions. On SMT level, function values are encoded by discriminating over the possible function values reaching a call site: when the concrete function is known, its effect is accounted for directly; when it is unknown (for example, a function parameter, or a closure loaded from storage), its behavioral predicates describe the effect. Our approach goes beyond, for example, Dafny, by supporting imperative first-class function which can modify state via Rust-style references and global variables, and leads to more efficient SMT encodings than separation logic because of the static separation of memory enabled by Move. We further extend MVP's *specification inference* tool to code with function values: given arbitrary higher-order Move code, weakest-precondition analysis automatically derives behavioral-predicate-based specifications, reducing the annotation burden and providing a validation pipeline for the new specification constructs.**

I. Introduction

The Move Prover (MVP) [1] is a formal verification tool for Move smart contracts that is designed to be used routinely during code development, with a finish time comparable to type checkers and linters. MVP is deployed at Aptos [2] for formal verification of core protocol logic such as staking, metering, code deployment, and auxiliary data structures that underpin the framework. It was originally developed for the Libra project at Meta, where where key properties were specified and proved for the Diem.

Recently, the Move language was extended with *higher-order functions* (HO) [3]. In *Move 2*, functions are first-class values: they can be passed as arguments, returned from functions, stored in struct fields, and persisted in global storage. Combined with Move's resource model, persisted function values give the developer a principled account of *dynamic dispatch*. Typical use cases on Aptos include callback-based asset flows, extensible asset managers, dispatchable permissions and metering, and generic data structure operations parameterized by a caller-provided function (e.g. fold, map, filter, custom comparators).

Higher-order functions are a known challenge for formal verification. They break the modular, static-dispatch specification style on which MVP was built. When a function is dispatched dynamically, its pre- and post-conditions cannot in general be looked up at the call site. Instead, the caller's specification must refer *abstractly* to the behavior of its function-value arguments, and the verifier must reconcile this abstract specification with whichever concrete function happens to reach the call site. For a production-oriented prover, this reconciliation must be fast, complete in the relevant cases, and free of the trigger instability that higher-order quantification easily introduces [4].

Contributions. This paper describes the extension of MVP to support higher-order Move, and the underlying extension of the Move specification language: Specifically:

- We extend Move specifications with *behavioral predicates*: $\mathsf{requires_of}\langle f\rangle(\bar{x})$, $\mathsf{aborts_of}\langle f\rangle(\bar{x})$, and $\mathsf{ensures_of}\langle f\rangle(\bar{x})$, which characterize the pre-/abort-/post-behavior of a function value f on arguments $\bar{x}$, enabling reasoning about function specifications as first-class citizens.
- We introduce *state labels* into Move specifications, which name intermediate memory states along a function's execution and connect them to the pre- and post-state via two-state predicates, as in $(..S \mid\!\sim \phi) \wedge (S.. \mid\!\sim \psi)$. This enables reasoning about sequential evolution of global state in predicates.
- We describe how MVP implements invocation of function values on top of SMT [5], [6], by discriminating over the possible types of function values flowing into the call site: if f is known (a ground constructor), the invocation is replaced by the effect of the concrete function; if f is not known (for example, a function-typed parameter or a closure loaded from storage), the behavioral predicates of f describe its effect.
- We extend MVP's *specification inference* tool to function-valued code. The tool computes weakest-preconditions over Move bytecode containing closures and dynamic dispatch, producing behavioral-predicate-based specifications automatically. This serves two purposes: it reduces the annotation burden for developers writing higher-order specs, and feeding inferred specs back into the

https://doi.org/

This article is licensed under a Creative Commons Attribution 4.0 International License

verifier provides a validation pipeline confirming that the encoding is internally consistent.

The remainder of the paper is organized as follows. Sec. II reviews Move on Aptos, introduces the running automated-market-maker (AMM) example, and illustrates common specification patterns using function values. Sec. III describes the encoding of behavioral predicates and state labels, and describes the Boogie encoding of dynamic dispatch in MVP. Sec. IV describes the extension of MVP's specification inference tool to higher-order code and demonstrates it on the AMM example. Sec. V discusses related work and concludes.

II. Using Function Values in Move Specifications

This section recalls the aspects of Move on Aptos that matter for the verification of higher-order programs, introduces an automated market maker (AMM) as a running example, and illustrates the common specification patterns our extension supports.

A. Move on Aptos

Move [7], [8] is a bytecode-verifiable programming language designed from the outset to coexist with formal verification. For the purposes of this paper, the relevant properties are:

- *Strong static typing with generics.* Every expression has a statically known explicit type; generic functions and structs are monomorphized for verification [1].
- *Reference semantics.* References are borrowed from local variables or from global storage and cannot be stored inside structs that go to global storage; this yields an alias-free memory model that makes verification tractable [1].
- *Type-indexed global storage.* Global state is a map keyed by (resource type, address) pairs: a struct with the `key` ability can be published, borrowed, and moved at an address via `T[addr]`, with linear ownership enforced statically. Specifications quantify over this memory directly, and `modifies` clauses name the footprint a function may touch, providing a frame condition.
- *Strongly-typed bytecode.* Move source is compiled to bytecode whose type- and memory-safety are checked before execution. MVP verifies this bytecode against specifications in the source.
- *Integrated specification language.* Move lets developers write pre-/post-conditions, abort conditions, data invariants, and global memory invariants in the same source file as the code they describe, or optionally, into separate files.

The Move Prover. MVP [1] is an automated formal verifier built into the Move toolchain. Developers annotate Move source with specifications — preconditions, postconditions, abort conditions, data invariants, and loop invariants — using a first-order specification language that shares Move's type system. MVP translates the annotated program to Boogie [9], an intermediate verification language, which Z3 [6] then discharges as SMT queries. The tool targets routine use during development: verification is designed to complete within seconds per function through careful SMT encoding and monomorphized, modular function summaries. Abort conditions deserve special note: multiple `aborts_if` clauses form a *biconditional* — the function aborts if and only if at least one clause holds — rather than the one-directional "if it aborts, then ..." familiar from partial-correctness Hoare logic.

Function values in Move 2. Move 2 extends the type system with *function types* $T_1 \times \cdots \times T_n \to T$ (written `|T1,...,Tn|T` in the concrete syntax) and allows values of such types to be constructed from concrete function symbols or from lambda expressions, possibly with captured arguments. In Move's type ability system, function values may be marked `copy`, `drop`, and `store`; a function value with `store` can be persisted to global storage. References may not be captured; only plain values may appear in a closure's payload, ensuring that captured data is independent of any borrow scope. This restriction is what makes the encoding in Sec. III-A tractable: each closure variant carries only typed values, with no aliasing between captures and the caller's memory. Invocation has the form `(expr)(args)` and is dispatched based on the closure reaching the call site.

B. Automated Market Maker Example

The running example throughout this paper is a simplified AMM module (Uniswap v2 [10]) in which the *pricing curve* is a function value stored in the pool. The pricing function has to satisfy a number of invariants: (i) the function never aborts, (ii) the output never exceeds the output reserve, (iii) the output is monotone in the input amount, and (iv) the product of the two reserves does not decrease after a swap (constant-product preservation). The `spec Pool` block below expresses these invariants using *behavioral predicates* and *state labels*, two constructs introduced by this paper and explained in detail after the listing.

```
struct Pool has key {
  reserve_x: u64, reserve_y: u64,
  // (reserve_in, reserve_out, amt) = out
  pricing: |u64, u64, u64|u64 has copy, store
}
spec Pool {
  reads_of<self.pricing> *;

  // Pricing must never abort.
  invariant ∀ S in *, ri: u64, ro: u64, a: u64:
    S |~ !aborts_of<self.pricing>(ri, ro, a);

  // Output never exceeds the output reserve.
  invariant ∀ S in *, ri: u64, ro: u64, a: u64:
    S.. |~ result_of<self.pricing>(ri, ro, a) ≤ ro
        ;

  // Monotonicity in the input amount.
  invariant ∀ S in *, ri: u64, ro: u64,
                          a1: u64, a2: u64:
    S.. |~ a1 ≤ a2 ⟹
      result_of<self.pricing>(ri, ro, a1)
              ≤ result_of<self.pricing>(ri, ro, a2
                );

  // Constant-product preservation.
  invariant ∀ S in *, ri: u64, ro: u64, a: u64:
    S.. |~ (ri + a)
```

```
            * (ro - result_of<self.pricing>(ri, ro,
                a))
                 ≥ ri * ro;
}
```

Each invariant is universally quantified over a state label S, ranging over all states reachable during execution in which the pool exists at some address. A *state label* such as S names a program point; the state at S comprises both the global memory state and the local state at that point. For the Pool invariants the local component is vacuous, since pricing takes only plain value arguments; the global component covers the resources declared by reads_of<self.pricing> *.

State labels appear in two predicate forms. A *single-state* predicate $S \mid\!\sim p$ evaluates p entirely in state S; the no-abort invariant uses this form because whether pricing aborts depends only on the pre-state of the call. A *two-state* predicate $S..\mid\!\sim q$ takes S as the pre-state of the pricing invocation and the resulting post-state as the second end; the output, monotonicity, and constant-product invariants use this form because the return value of pricing is only defined after execution from S. More generally, $S_1..S_2 \mid\!\sim q$ names both ends explicitly; $..S \mid\!\sim q$ fixes S as the post-state, with the enclosing context supplying the pre-state.

The behavioral predicates aborts_of, result_of, and (elsewhere) ensures_of and requires_of, take a function value and its arguments and evaluate to the corresponding aspect of that call's behavior. Without behavioral predicates, specifying a higher-order function would require inlining the callee's conditions at every call site, leading to a combinatorial explosion in specification size; behavioral predicates make reasoning about function values modular and abstract. Using them in invariants as in the example creates (i) a requirement for any code creating a pool to pass only compliant function values, and (ii) a guarantee for any code calling a pool function that the requirements are met.

Swapping. The major functionality of the pool is to facilitate swap (exchange) of one asset with another.

```
fun swap(pool: &mut Pool, amt: u64): u64 {
  let out = (pool.pricing)(
              pool.reserve_x, pool.reserve_y, amt
                  );
  pool.reserve_x += amt; pool.reserve_y -= out;
  out
}
spec swap {
  // Can only abort on reserve overflow.
  aborts_if pool.reserve_x + amt > MAX_U64;
  // Outcome is determined by pricing at pre-state
      reserves.
  ensures result = old(result_of<pool.pricing>(
            pool.reserve_x, pool.reserve_y, amt));
  // Preserves product in pool.
  ensures pool.reserve_x * pool.reserve_y
           ≥ old(pool.reserve_x * pool.reserve_y)
               ;
}
```

The specification of swap can be verified by MVP because of the invariants guaranteed for the (dynamic) pricing function. In this case, we only illustrate the aborts condition and product preservation. For aborts, pricing itself is non-aborting, so only the overflow conditions remain. (Notice MVP would flag any aborts conditions not covered). For product preservation, the constraint can be derived as well.

Pricing variants. At the time a pool value is constructed, the invariants for the pricing function need to be verified. Obviously this is more difficult than the application side, since now properties of functions need to be proven. In reality, those proofs will often need custom solutions, but for the given example, MVP can handle some cases gracefully. First, we define a function which computes the so-called *constant product*, which is a pricing function describing a direct swap without fees; however, rounding errors can lead to a loss in returned assets:

```
fun product(ri: u64, ro: u64, a: u64): u64 {
  let num = (ro as u128) * a;
  let den = (ri as u128) + a;
  if (den = 0) 0 else (num / den) as u64
}
```

When constructing a pool using Pool{.., pricing: product}, MVP is able to discharge all verification conditions.

The next example uses a pricing function which charges a configurable fee:

```
struct Fee has key {bps: u64}
fun with_fee(o: address,
             ri: u64, ro: u64, a: u64): u64 {
  let b = (a as u128)
     * (10000 - Fee[o].bps as u128) / 10000;
  product(ri, ro, b)
}
```

In order to construct a pricing function we need to curry the address for the fee resource and capture it in a closure, which can be done in Move as below:

```
let o: address;
Pool{
  ..,
  pricing: |ri, ro, a| with_fee(o, ri, ro, a)
}
```

MVP detects that with_fee violates the requirement to not abort. (Abort happens if the resource Fee[o] does not exist.) Guarding the definitions with checks and using a default fee fixes this and makes verification succeed.

In practice, pricing functions such as with_fee would be annotated with pragma opaque — a directive that hides the function body from callers, so they reason only from the declared spec rather than the implementation, keeping verification costs tractable at scale. However, opaque creates a subtle dependency: the behavioral predicate aborts_of<f> used in the data invariant is derived solely from the declared aborts_if conditions of f. If no aborts_if is specified, the prover treats the function as non-aborting — aborts_of<f> defaults to false — and the no-abort invariant is discharged vacuously, even if the implementation can in fact abort. The developer must therefore supply complete aborts_if conditions for any function value used in a behavioral invariant, or risk unsound verification. Deriving these conditions correctly is non-trivial, especially for functions that read global state under multiple guards. MVP's *specification inference* tool

addresses this directly: weakest-precondition analysis of the function body automatically produces the correct `aborts_if` conditions, ensuring the declared spec is complete before the function is used in a behavioral invariant. Sec. IV demonstrates this on the AMM pricing functions.

The AMM example shows that non-trivial invariants can be specified for function values, and that the prover is capable of verifying them on construction and caller side. What has been shown for data field invariants carries over to function parameters and function call sites. The invariants presented constitute non-linear arithmetic problems, which are hard to solve. In practice, Move 2 proof hints are often needed to support reliable verification of problems of this kind.

C. Filter-style Loops

Behavioral predicates are not limited to struct invariants; they appear naturally in loop invariants and function specs for higher-order collection operations. As a minimal illustration, consider a generic `find` which returns the first index of `v` whose element satisfies a predicate `pred`, or `len(v)` when no such element exists. A spec-level helper `no_match_before` captures the recurring "no earlier element has matched" pattern:

```
fun find<T>(v: &vector<T>,
            pred: |&T|bool has copy + drop): u64 {
  let i = 0;
  let n = v.length();
  while (i < n) {
    if (pred(v[i])) return i;
    i += 1;
  } spec {
    invariant i ≤ n;
    invariant no_match_before(v, pred, i);
  };
  n
}
spec find {
  pragma opaque;
  ensures result ≤ len(v);
  ensures no_match_before(v, pred, result);
  ensures result < len(v)
            ⟹ result_of<pred>(v[result])
}
spec fun no_match_before<T>(v: vector<T>,
                pred: |&T|bool, end: u64): bool {
  ∀ j in 0..end: !result_of<pred>(v[j])
}
```

`no_match_before` is an ordinary spec function, but its body calls the behavioral predicate `result_of<pred>` on each element — a predicate-parametric abstraction is thus first-class in Move and can be reused across loop invariants and post-conditions. The function's post-condition characterises `result` entirely through the same evaluator: either `result` is a valid index whose call is the first to match, or `result` $=$ `len(v)` and no element matches at all. No concrete predicate is ever named in either the body or the spec; a caller instantiates `pred` with any function value, and the specification is discharged through its behavioral contract.

To illustrate the proof obligation at the caller, consider specialising `find` with an inline lambda whose behavioral contract is given by an inline `spec` clause:

```
fun find_zero_lambda(v: &vector<u64>): u64 {
  find(v, |x| x = 0
           spec { ensures result = (x = 0); })
}
spec find_zero_lambda {
  ensures result ≤ len(v);
  ensures result < len(v) ⟹
    v[result] = 0 &&
    (∀ j in 0..result: v[j] ≠ 0);
  ensures result = len(v) ⟹
    (∀ j in 0..len(v): v[j] ≠ 0);
}
```

Since `find` is opaque, the prover reasons from `find` 's spec alone. At the match index, `find` guarantees $\mathsf{result_of}\langle pred\rangle(v[result])$, which — by the lambda's inline spec — reduces to `v[result]` $=$`0`. For earlier indices, `no_match_before` constrains $\mathsf{result_of}\langle pred\rangle(v[j])$ to be false for every index `j` before `result`, which reduces to $v[j] \neq 0$. The prover discharges all three `ensures` clauses of `find_zero_lambda` without ever inspecting the lambda body.

D. More About State Labels

The Pool invariants in Sec. II-B used *universally* quantified state labels: every state in which the pool exists must satisfy the pricing invariants. State labels have a second, equally important use: *existentially* quantifying over an intermediate state to express that a computation proceeds through a specific intermediate point. Consider a higher-order function that sequences two stateful operations on a shared value:

```
fun followed_by(f: |&mut A|, g: |&mut A|, x: &mut
    A) {
  f(x); g(x)
}
spec followed_by {
  ensures ∃ S in *:
    (..S |~ ensures_of<f>(x)) &&
    (S.. |~ ensures_of<g>(x))
}
```

The existentially quantified label `S` witnesses the intermediate state between the two calls. Since no `reads_of` is declared for `f` or `g`, they are assumed to have no global memory footprint; the local component of the state at `S` is therefore the updated value of `x`, and the global component is vacuous. The conjunct $..S \mid\sim \mathsf{ensures_of}\langle f\rangle(x)$ asserts that the postcondition of `f` holds from the pre-state of `followed_by` up to `S`; the conjunct $S.. \mid\sim \mathsf{ensures_of}\langle g\rangle(x)$ asserts that the postcondition of `g` holds from `S` to the final post-state. Crucially, the post-state of `f` automatically becomes the pre-state of `g`, without any explicit threading of intermediate values through the specification.

State labels are not restricted to higher-order functions. When `f` and `g` are concrete named functions, the same pattern allows a wrapper to be specified by composing the behavioral predicates of its callees, avoiding duplication of their postconditions.

III. Encoding

We describe the implementation of function values as a translation into an imperative intermediate verification language (IVL) inspired by Boogie [9]. The encoding is given

at the IVL level because this is where the interesting design decisions become visible — the subsequent step to pure SMT is straightforward and uses standard techniques. The fragment of the IVL used in this section is familiar: procedures with **havoc**, **assume**, **assert**, pure functions, *algebraic datatypes* (tagged unions with selectors, like Rust enums), universal *axioms* with *triggers*, and polymorphic map types. State is represented by Boogie-style maps; in particular, each resource type τ yields a map $M_\tau \in \mathsf{address} \to \tau$, with address being the domain of addresses in Move (256-bit integers). The representation is similar to that in the earlier MVP paper [1].

In the following we focus on the aspects specific to function values: the representation of function values (Sec. III-A), the memory access and modification sets used to define the invocation schema (Sec. III-B), the encoding of invocation of function values (Sec. III-C), the encoding of behavioral predicates via axioms (Sec. III-D), and finally the encoding of intermediate state labels (Sec. III-E). Everything in this section operates *after* monomorphization, so type parameters have been specialized and the set of closure, parameter, and field variants reaching a given function type is known statically [1].

A. Representing Function Values

For each function type $\tau = (T_1, \ldots, T_n) \to T$ that occurs in the monomorphized program, we create a datatype in the IVL which represents all the potential *sources* of function values in a given program. To this end, MVP's mono-analysis records three sets: the set $\mathcal{C}_\tau$ of concrete functions f constructing closures of type τ, the set $\mathcal{P}_\tau$ of function-typed parameters of verification targets, and the set $\mathcal{F}_\tau$ of storable function-typed struct fields whose type is τ. The union of these three sets is materialized as an algebraic datatype in the IVL — one constructor per variant:

$$\begin{array}{l}\textbf{datatype}\ \tau\ \{\\ \quad C_f(\overline{c:T}), \ldots, P_{f,x}(), \ldots, F_{\sigma.x}(n:\mathbb{N}), \ldots\\ \}\end{array}$$

Here $\overline{c:T}$ are the types of the values captured by closure f; $P_{f,x}$ is the parameter variant for function-value parameter x of function f; and $F_{\sigma.x}$ is the field variant for struct field x of struct type σ (we use σ here to avoid confusion with the state-label variable S introduced in Sec. III-E). Parameter variants are nullary because, in a well-typed verification condition (VC), the value of an entrypoint function-typed parameter is a skolem constant pinned to its variant at entry; no runtime payload is required. Field variants carry an integer discriminator n so that two locations in memory holding distinct field values are not forced to be equal — a datatype with a single nullary constructor would otherwise collapse all field values to one.

Notice that this representation captures *all* possible functions of type τ relevant for a given VC. So we do not actually need to deal with an arbitrary number of function values when it comes to higher-order function verification, but only those which can be derived from the program. The representation is further narrowed if the function value of a given type is not stored in a structure which ends in global memory. Storing a function value in a struct field introduces dynamic dispatch: two different field locations may hold different closures, and the discriminator n in $F_{\sigma.x}(n)$ ensures the solver treats them as distinct. Nevertheless, we are able to pinpoint a specification to a particular $F_{\sigma.x}(n)$, as illustrated by the AMM example (Sec. II-B).

B. Computing Memory Accesses

Invoking a function value requires reasoning about which memory maps a given function type τ may read ($\mathcal{M}_\tau$) and which locations it may modify ($\Delta_\tau(\overline{p})$); we define both here before presenting the invocation schema. To recall, in the Move language, persistent global storage, or memory, can be accessed via type indexing. For instance, if `R` is a resource type, we write `&mut R[addr]` in the language to obtain a mutable reference to the memory which can then be subsequently mutated.

MVP's representation of memory uses type-indexed maps $M_\rho \in \mathsf{address} \to \rho$, with ρ a resource type (a `struct` in Move with the `key` ability). In the imperative procedures of the IVL, those maps are implicitly available as global variables. But in pure expressions like behavioral predicates, they need to be made explicit as parameters; this is why $\mathcal{M}_\tau$ appears as an explicit argument to behavioral predicates throughout this section.

A few definitions for memory maps are needed. Note that these are definitions on the meta level of the description here and not part of the IVL; they are used to describe pseudo code in terms of the IVL.

First, with $\mathcal{M}_R = \{M \in \mathsf{address} \to \rho \mid \rho \in R\}$ we denote a set of memory maps for the resource types in R. The R can be omitted if it is clear from the context. For convenience, with $\mathcal{T}(\mathcal{M})$ the set of all types ρ used in the memory maps of $\mathcal{M}$ is described. With $\mathcal{M}_R \downarrow S$, where $S \subseteq R$, the projection $\{M \in \mathcal{M}_R \mid \rho(M) \in S\}$ is denoted.

In addition to memory, we also need to reason about modifications. A modification is a set of pairs of a type and an expression of the IVL of type address, $\Delta \in \mathbb{P}(\mathcal{T} \times \mathcal{E})$. We write $\Delta(\overline{p})$ for a set of modifications built on expressions referencing parameters in $\overline{p}$.

General Access. Memory access $\mathcal{M}_\tau$ is computed as the union of all the accesses in the variants of τ, $\mathcal{M}_{C_f} \cup \mathcal{M}_{P_{f,x}} \cup \mathcal{M}_{F_{\sigma.x}}$. For a closure, $\mathcal{M}_{C_f}$ can be statically derived from the code. For parameters and fields, the information is declared in the spec block:

```
spec foo(f: |T|S, g: |T|S) {
  reads_of<f> R; // f can read resource R
  reads_of<g> *; // g can read any resource
                 // in scope of the VC
}
```

For the wildcard operator `*`, all memory which is read or written as part of the currently generated VC is included, and in addition an abstract memory map for any 'unknown' memory the function may depend on. The unknown memory is needed for a sound encoding. For instance, if we verify

function `foo` above in a context where there is no memory involved anywhere, we must not assume that `g(x)` delivers the same value in different states, as it may depend on this unknown additional memory. However, we can still know that the specification of `g` holds in every possible state.

Write Accesses. Similarly to general access, the modifies set Δ_τ is computed as the union of all the modifications in the variants of τ, $\Delta_{C_f} \cup \Delta_{P_{f,x}} \cup \Delta_{F_{\sigma.x}}$.

For closures, Δ_{C_f} is derived from the code and, if present, the `modifies R[a]` declaration in the language. These existed before higher-order functions, for specifying what functions modify.

For parameters and fields, the information is declared in the spec block, where for the `modifies_of<f>` clause, the parameters of the function value itself are passed:

```
spec foo(f: |address|) {
  // f can modify resource R[a]
  modifies_of<f>(a: address) R[a];
}
```

If no `modifies_of` is declared, it is assumed that the function does not modify global memory.

Notice that a wildcard (`modifies_of<f> *`) is not supported, since the invocation of `f` would render the memory as arbitrary (no effective frame condition). This is a notable restriction on what we can specify with higher-order functions in Move and is left for future work.

C. Invoking Function Values

When a function value is invoked, we are looking at either a concrete closure value C_g or a parameter or field selection as described above. In the former case, one can simply delegate to calling the actual underlying function f, by composing the captured and provided arguments into a full argument list.

In the other cases of parameters and field selections, we use the behavioral predicates as defined for the function value, using the standard schema for encoding a call to a Move function via the function's specification. This reduces the problem of invocation to how those behavioral predicates are encoded. In the procedure below, $\mathcal{M}_\tau$ denotes the projection of the current global memory state to the resource types in τ's access set (as computed in Sec. III-B).

$$
\begin{array}{l}
\textbf{proc}\ \mathit{invoke}_\tau(x:\tau, \overline{p:T})\ \mathsf{returns}\ (\overline{r:T}) \\
\quad \textbf{if}\ x\ \mathsf{is}\ \ C_f(\overline{c}) \\
\qquad \overline{r} := \mathsf{call} f(\overline{c}, \overline{p}) \\
\quad \textbf{else} \\
\qquad \textbf{assert}\ \mathit{requires}_\tau(x, \mathcal{M}_\tau, \overline{p}) \\
\qquad \textbf{if}\ \ \mathit{aborts}_\tau(x, \mathcal{M}_\tau, \overline{p}) \\
\qquad\quad \mathit{abort_flag} := \textbf{true} \\
\qquad \textbf{else} \\
\qquad\quad \textbf{let}\ \mathcal{M}'_\tau := \mathcal{M}_\tau \\
\qquad\quad \textbf{havoc}\ \Delta_\tau(\overline{p}),\ \overline{r} \\
\qquad\quad \textbf{assume}\ \mathit{ensures}_\tau(x, \mathcal{M}'_\tau, \mathcal{M}_\tau, \overline{p}, \overline{r})
\end{array}
$$

Here $\textbf{let}\ \mathcal{M}'_\tau := \mathcal{M}_\tau$ freezes the pre-invocation memory into $\mathcal{M}'_\tau$, while $\mathcal{M}_\tau$ becomes the post-invocation state after **havoc**. We therefore pass $\mathcal{M}'_\tau$ as the pre-state and $\mathcal{M}_\tau$ as the post-state to $\mathit{ensures}_\tau$ (note that $\mathcal{M}'$ consistently denotes the pre-state throughout this section). $\textbf{havoc}\ \Delta_\tau(\overline{p})$ is shorthand for havocing, for each $(\rho, a) \in \Delta_\tau(\overline{p})$, the entry $M_\rho[a]$ of the type-indexed memory map for ρ. *abort_flag* is a prover-internal boolean global that signals an abort to the enclosing verification condition, consistent with how MVP models Move's abort semantics throughout the IVL. The **if** branch reduces to a standard function call $\mathsf{call} f(\overline{c}, \overline{p})$, which MVP encodes using the same **assert/havoc/assume** schema as the **else** branch, specialized to f's declared spec. When f is opaque, the two branches are in fact semantically equivalent: an opaque call is already compiled to this schema driven by f's spec, matching what the **else** branch does for the abstract variant. The **else** branch therefore introduces no new encoding machinery; it applies the existing opaque-call schema, driven by the behavioral predicates of the unknown variant x rather than a concrete function's spec.

D. Behavioral Predicates

Behavioral predicates are derived by switching over the variants of the function value representation τ, similarly to the invoke_τ procedure above, delegating extraction of the underlying predicates to per-variant functions.

Below, the definition for ensures is given. The other predicates are defined similarly. Specific to ensures is that it needs to take care of the frame condition: the caller havoced the full modification set, but only a subset of that is constrained by the given variant. The handling needs to account for the fact that modification addresses are symbolic expressions that may alias.

In what follows, we write $\mathit{pred}_\tau[\mathcal{M}', \mathcal{M}](\ldots)$ with square brackets for the memory pair: $\mathcal{M}'$ is the pre-state and $\mathcal{M}$ the post-state, following the convention established in Sec. III-C. These memory arguments must be passed explicitly to pure functions, whereas they are implicit globals in IVL procedures. We write Δ_x for the modification set of the specific variant of x — that is, Δ_{C_f} when $x = C_f(\overline{c})$, $\Delta_{P_{g,y}}$ when $x = P_{g,y}()$, and $\Delta_{F_{\sigma.x}}$ when $x = F_{\sigma.x}(n)$. We write $\mathcal{M}'(\rho)$ for the map M_ρ in the pre-state, and $\mathcal{M}(\rho)$ for the map M_ρ in the post-state.

$$
\begin{array}{l}
\textbf{fun}\ \mathit{ensures}_\tau[\mathcal{M}', \mathcal{M}](x:\tau, \overline{p:T}, \overline{r:U}) : \mathsf{bool} \\
\quad \bigwedge_{(\rho,a)\in\Delta_\tau} (\\
\qquad \textbf{if}\ \rho \notin \mathcal{T}(\Delta_x) \\
\qquad\quad \mathcal{M}(\rho) = \mathcal{M}'(\rho) \\
\qquad \textbf{else} \\
\qquad\quad (\bigwedge_{(\rho,a')\in\Delta_x} a' \neq a) \\
\qquad\qquad\quad \Rightarrow \mathcal{M}(\rho)[a] = \mathcal{M}'(\rho)[a] \\
\quad) \wedge \mathit{ensures}_x[\mathcal{M}'_x, \mathcal{M}_x](p, r)
\end{array}
$$

The relational representation of a function via ensures does not guarantee proper functional behavior. This is achieved by introducing result_τ as an uninterpreted function, connected via an axiom:

$$\textbf{fun}\ result_\tau[\mathcal{M}', \mathcal{M}](x : \tau, \overline{p : T}) \to \overline{U}$$
$$\textbf{axiom}\ \forall x, \overline{p}, \overline{r} \bullet$$
$$ensures_\tau[\mathcal{M}', \mathcal{M}](x, \overline{p}, \overline{r}) \Leftrightarrow \overline{r} = result_\tau[\mathcal{M}', \mathcal{M}](x, \overline{p})$$

The actual conditions extracted depend on the variant. For closures, they are directly extracted from the spec block. For parameters and fields, we introduce uninterpreted functions.

$$\textbf{fun}\ ensures_{C_f(\overline{c})}[\mathcal{M}', \mathcal{M}](\overline{p : T}, \overline{r : U}) : \texttt{bool}$$
$$\langle extract\ from\ spec\ block\ of\ f(\overline{c}, \overline{p})\rangle$$
$$\textbf{fun}\ ensures_{P_{f,x}}[\mathcal{M}', \mathcal{M}](\overline{p : T}, \overline{r : U}) : \texttt{bool}$$
$$\textbf{fun}\ ensures_{F_{\sigma.x}}[\mathcal{M}', \mathcal{M}](\overline{p : T}, \overline{r : U}) : \texttt{bool}$$

The meaning of the uninterpreted functions is determined by how they are injected into a VC via the regular mechanism by which pre/post conditions and invariants are handled. Consider a precondition like `requires aborts_if<param>(x)`, where `param` is a function-value parameter of a function `foo`. This will be injected in the VC as $\textbf{assume}\ \neg\ aborts_{P_{foo,param}}(x)$. Similarly, for a data invariant of a field an assumption will be generated when the field is selected.

E. Intermediate State Labels

Recall from Sec. II that a state label `S` denotes a program point. Formally, a *state* at label S is a pair $(\mathcal{M}^S, \overline{v}^S)$ consisting of a memory snapshot and a tuple of local values. The memory component $\mathcal{M}^S$ has the same type as the ambient memory: a tuple of type-indexed maps $M_\rho \in \texttt{address} \to \rho$ for each resource type ρ in scope at that program point, quantified existentially in the IVL. The local component $\overline{v}^S$ captures values of mutable-reference parameters at the intermediate point: for a closure over a mutable reference of type T with no global memory effects, $\overline{v}^S$ is a single value $x^S : T$, and $\mathcal{M}^S$ is vacuous (the function touches no global resources). When a closure both accesses global memory and takes mutable references, both components are non-trivial. Here we focus on the global-memory case (vacuous $\overline{v}^S$); Sec. III-F notes how the mutable-reference case is handled.

Consider the following specification using intermediate state labels:

```
struct R(u64) has key;
spec foo(f: |address|, g: |address|, a: address) {
  ensures ..S |~ ensures_of<f>(a)
  ensures S |~ R[a].0 > 0;
  aborts_if S |~ aborts_of<g>(a);
  ensures S.. |~ ensures_of<g>(a)
}
```

As outlined in the last section, we want to be able to extract behavioral predicate functions from this specification. For the ensures we can come up with the following, which simply replicates part of the spec:

$$\textbf{fun}\ ensures_{C_{foo}}[\mathcal{M}', \mathcal{M}](a)$$
$$\exists\, \mathcal{M}^S \bullet\ ensures_\tau(f, \mathcal{M}', \mathcal{M}^S, a) \wedge R[\mathcal{M}^S][a].0 > 0 \wedge$$
$$ensures_{\tau'}(g, \mathcal{M}^S, \mathcal{M}, a)$$

For the aborts which happens at the intermediate state `S`, the state `S` needs to be first established before the aborts condition can be checked for g. To achieve this, all ensures conditions which reach `S` are combined with the aborts condition, as in:

$$\textbf{fun}\ aborts_{C_{foo}}[\mathcal{M}', \mathcal{M}](a)$$
$$\exists\, \mathcal{M}^S \bullet\ ensures_\tau(f, \mathcal{M}', \mathcal{M}^S, a) \wedge R[\mathcal{M}^S][a].0 > 0 \wedge$$
$$aborts_{\tau'}(g, \mathcal{M}^S, a)$$

This is semantically sound because we can assume inductively that the function *foo* has successfully verified. For a successfully verified function, where $\overline{A}$ are the `aborts_if` conditions and $\overline{P}$ the `ensures` conditions, it holds:

$$\bigvee \overline{A} \vee \bigwedge \overline{P}$$

That is, either the function aborts under one of the given conditions, or all of its ensures conditions hold.

In general, since the relation between state labels spawned by predicates of the form $S_1..S_2 \mid\!\sim p$ must be acyclic, one can extract from $\overline{P}$ those conditions which lead into $\mathcal{M}^S$, and from $\overline{A}$ those which start from state $\mathcal{M}^S$. Given this, the aborts condition can be synthesized using the following scheme:

$$\bigwedge \overline{P}_{..\mathcal{M}^S} \wedge (\bigvee \overline{A}_{\mathcal{M}^S} \vee \bigwedge \overline{P}_{\mathcal{M}^S..\mathcal{M}^T} \wedge (\bigvee \overline{A}_{\mathcal{M}^T} \vee \bigwedge \ldots))$$

Here, $\overline{P}_{..\mathcal{M}^S}$ establish state $\mathcal{M}^S$ in which we check for the aborts condition $\overline{A}_{\mathcal{M}^S}$, or recurse over the remaining $\overline{P}$ and $\overline{A}$.

F. Implementation Notes

A straightforward implementation following the conceptual description above leads to SMT timeouts on non-trivial examples. The root cause is that the presentation writes behavioral predicates as IVL functions with explicit bodies ($\textbf{fun} \ldots = body$), and Boogie unfolds function bodies eagerly: any quantifier inside a body — common in arithmetic identities or prelude vector axioms — spawns trigger instantiations at every call site, causing combinatorial blowup in the SMT search.

We overcame this by adopting a pattern that decouples the *definition* of behavioral predicates from their *use*: declare each predicate as an *uninterpreted* function and connect it to the per-function specification via axioms with explicit, carefully chosen triggers. This pattern is not specific to higher-order functions; it is a general Boogie encoding discipline for modular specifications that keeps SMT instantiation under control. The remaining implementation choices are natural refinements of applying this pattern consistently:

- *Behavioral predicates are uninterpreted functions connected by axioms, not Boogie functions with explicit bodies.* The implementation declares $ensures_\tau$ as an uninterpreted Boogie function and connects it to the per-function spec via an axiom of the form $(\forall \ldots :: eval_call \Leftrightarrow rhs)$ with an explicit trigger pattern. Axioms with explicit triggers fire only when the trigger pattern matches a ground term in the proof context, keeping instantiation under control.
- *Per-variant trigger specialization.* The three variant kinds receive structurally different axioms. Closure variants are triggered by the constructor application $C_f(c_0, \ldots, c_K)$

with capture variables bound by the quantifier. Function-parameter variants are triggered by the nullary constructor $P_{f,x}()$. Struct-field variants are triggered by the evaluator itself with the variant pinned via an f is $F_{\sigma.x}$ guard, because field values at call sites are typically opaque datatype values loaded from memory rather than literal constructors. This specialization ensures that the SMT solver instantiates each axiom only where the corresponding variant is syntactically present in the context.

- *Constraining state-label conjuncts are dropped from the existential.* The conceptual behavioral predicate for an `aborts_if` at an intermediate state $\mathcal{M}^S$ includes *all* ensures-with-S conjuncts that reach $\mathcal{M}^S$ (including constraining ones such as $R[\mathcal{M}^S][a].0 > 0$). The implementation restricts the existential body to the *defining* conjuncts — those introduced by label-defining operations like $\mathsf{ensures_of}\langle f\rangle(\bar{x})$, $\mathsf{result_of}\langle f\rangle(\bar{x})$, or the publish/remove/update builtins. By the validity invariant $\bigvee \overline{A} \vee \bigwedge \overline{P}$, the constraining conjuncts hold at the witness chosen during verification of `foo`, so dropping them is sound and keeps the existential body small.
- *State-label existential is flat, not recursive.* The recursive scheme above is logically equivalent to a single flat $(\exists\, \mathcal{M}^S, \mathcal{M}^T, \ldots \;::\; \ldots)$ wrapping the conjunction of defining fragments and the kind-specific clauses. The implementation uses the flat form, reducing the size of emitted Boogie and avoiding the additional SMT quantifier nesting that the recursive scheme would introduce.

IV. Specification Inference for Higher-Order Code

A. Spec Inference Overview

MVP's *specification inference* based on weakest-precondition (WP) analysis [11] was extended to handle function-valued code. WP inference takes arbitrary Move code and, for each function, computes conditions that are both necessary and sufficient for the implementation's behavior. The WP of a higher-order function is expressed in terms of `ensures_of` and `aborts_of` on its function-valued arguments rather than inlining their bodies; without this modular abstraction, inferred specs would be impractically large. This extension serves two purposes, demonstrated in the following subsections: reducing the annotation burden on developers writing higher-order specs (Sec. IV-B), and providing a validation pipeline that confirms the encoding is internally consistent (Sec. IV-C).

B. AMM Compliance Case Study

Recall from Sec. II-B that the `Pool` struct carries a `pricing` closure constrained by a no-abort invariant: the pricing function must not abort for any inputs in any state. Two concrete pricing implementations were shown: `constant_product_with_fee`, which is compliant (it never aborts), and `with_fee_non_compliant`, which is not (it aborts when the owner's `Fee` resource is absent). Spec inference makes this compliance distinction machine-readable without any manual annotation.

Bottom-up: exposing abort behavior. Given the implementation of `with_fee_non_compliant` with no written spec, WP inference derives its abort conditions automatically:

```
spec with_fee_non_compliant { // [inferred]
  aborts_if !∃<Fee>(owner);
  aborts_if Fee[owner].bps > 10000;
  aborts_if reserve_in + effective_in = 0;
  ...
}
```

The inferred abort condition — that the function aborts when the `Fee` resource for the owner is absent — directly contradicts the Pool no-abort invariant, which requires that `self.pricing` never aborts for any inputs in any state. When this function is used to construct a `Pool`, MVP reports a pack-time invariant violation — a precise, counterexample-driven diagnosis of non-compliance.

Constructor-side: compliance as a conditional postcondition. Running WP inference on `create_pool`, which accepts an arbitrary `pricing` argument, makes the compliance obligation explicit as a guard in the postcondition:

```
spec create_pool { // [inferred]
  ensures (∀ x: u64, y: u64, z: u64:
              !aborts_of<pricing>(x, y, z))
      ⟹ publish<Pool>(addr, Pool{..., pricing});
}
```

The pool is published only when the guard holds, i.e. only when `pricing` satisfies the no-abort invariant. For the non-compliant constructor, the guard can never be satisfied, so the desired postcondition is vacuous — the pool is never published. For the compliant constructor, the guard always holds and the pool is reliably created. In both cases, spec inference makes the compliance condition machine-readable without any manual annotation.

C. Validation Pipeline and Test Coverage

Inferred conditions are labeled `[inferred]` and feed directly back into MVP for verification, creating a pipeline: code $\rightsquigarrow$ code+specs $\rightsquigarrow$ VC, with verification expected to succeed. Tab. I summarizes the inference test suite exercising behavioral predicates across all HOF patterns introduced in this paper. All categories pass end-to-end, confirming that the encoding of behavioral predicates and state labels is internally consistent and that inferred specifications are tight enough for the verifier to discharge.

V. Discussion and Conclusion

A. Related work

Higher-order programs with side effects have been a long-standing concern in program verification. Our approach to behavioral predicates descends from the relational tradition of VDM [12], Z [13], [14], and the B method [15], in which operations are specified by pre/post relations over an explicit state; state labels generalize this to intermediate snapshots and make the relational flavor composable.

TABLE I: Inference test categories that exercise behavioral predicates.

Category	Representative cases
Direct calls	Single and chained calls to a spec'd helper; inferred specs use `ensures_of` and `aborts_of` to abstract over the callee's effect rather than inlining its body.
Mutual recursion	Mutually recursive `is_even`/`is_odd`; inference breaks the cycle by phrasing each spec in terms of the other's behavioral predicate.
Higher-order parameters	A function that takes a closure and applies it; the inferred spec abstracts the closure via `ensures_of`/`aborts_of` on the parameter, with no commitment to a particular implementation.
Stored-closure dispatch (enum)	Calculator with an enum variant carrying a closure field; dispatch through the variant is summarised by behavioral predicates on that field.
Stored-closure dispatch (struct field)	Vault with a `Strategy` closure stored in a struct field on Aptos framework types; inference emits `modifies_of` for the field together with the field-form behavioral predicates.
Pre/post state labels	`move_to`, `move_from`, mutable resource indexing; inferred specs distinguish `@pre` from post-state on the memory the predicate observes.
Intermediate state labels	Sequenced state-modifying calls (e.g. remove-then-publish); inferred specs introduce existentially quantified intermediate labels and pin them to defining behavioral predicates.

Among contemporary verifiers for imperative code with higher-order functions, Verus [16] offers function-value specifications via *spec closures* and phantom ghost state; the approach is state-passing, with state threaded as an extra argument to specifications rather than quantified over directly. Our treatment is more directly relational: state labels denote memory snapshots and are bound by quantifiers in the surface syntax, avoiding the encoding cost of an explicit state argument and matching the style in which Move developers already write pre/post specifications.

F* [17] likewise verifies higher-order, effectful code, internalizing the WP calculus in its type system via Dijkstra monads [18]. Both systems support pre/post specifications, quantifiers, behavioral abstraction over function values, and structured proof guidance — in Move via `proof` blocks, `lemma` declarations, and `calc`, `split`, `apply` statements. One major difference is that F* is based on separation logic, while Move uses traditional frame conditions via modifies declarations. While F* is more expressive via separation logic, it misses the syntactic, compile-time separation of memory via Move's resource and reference model, potentially leading to significantly more complex verification problems, as memory separation has to be reasoned about by the SMT solver.

Dafny [19] is a close cousin of MVP: it has a similar specification model of pre/post conditions, higher-order functions, and behavioral predicates. However, Dafny does not allow function values to modify state. This would be a severe restriction for Move, since it is a common usage pattern. Moreover, Move's resource-separated global memory and alias-free mutable reference model is not available in Dafny, similar as in F*. While both languages support state labels, Move state labels as described here can be used in abstract specifications via universal and existential quantification.

Lean 4 [20] is a dependently-typed interactive theorem prover in the tradition of older similar systems [21], [22], supporting higher-order functions. Verification is tactic-driven and kernel-checked. Lean does not directly work on a programming language as MVP does, but requires the user to create an encoding of the language inside its meta logic. For modeling of stateful systems, one would typically use monads, though any other model can be simulated as well. In contrast, MVP is directly integrated with the Move language and leverages Move's specific memory and reference model. Furthermore, MVP aims at automated SMT-based verification, with optional proof scripts.

In the smart-contract setting, dynamic dispatch is a central challenge for Solidity verifiers. Certora [23] handles dispatch on Solidity by enumerating the possible callees of a virtual call (derived from the contracts in scope) and generating a case-split proof obligation per callee; summaries can be supplied by the user to abstract external calls. Other SMT-based Solidity tools [24], [25], [26] handle dynamic dispatch through similar case-splitting, with varying degrees of abstraction. MVP is more general: a call site's dispatch is explicit in the closure's datatype, and our encoding lets the prover automatically fall back from case-split (when the concrete function is known) to behavioral-predicate reasoning (when only the function type is known), without the user having to structure the specification differently.

B. Future Work

The current encoding enumerates all closure variants reaching a given call site; scaling to programs where the variant set is large or open (e.g. dynamically loaded modules) remains future work. Proof automation for non-linear arithmetic arising in pricing formulas and similar domains currently relies on user-supplied proof hints; integrating portfolio arithmetic decision procedures is a natural direction. The spec inference pipeline produces behavioral-predicate-based specifications from arbitrary code; extending it to synthesize loop invariants involving function values automatically would reduce the remaining manual annotation burden.

C. Conclusion

We described the extension of the Move Prover to Move 2's higher-order functions and dynamic dispatch. The extension rests on two language additions — behavioral predicates and state labels — and on a Boogie encoding that discriminates closure invocations on the function-value variant reaching the call site. The design preserves MVP's production-oriented cost model — routine, predictable, counterexample-driven

verification — in the presence of dynamic dispatch. We further extended MVP's specification inference tool to closures and dynamic dispatch: given arbitrary higher-order Move code, the tool now computes weakest-precondition-based specifications automatically, turning the question of whether a function value complies with a behavioral invariant into an automated verification problem.

References

[1] D. L. Dill, W. Grieskamp, J. Park, S. Qadeer, M. Xu, and J. E. Zhong, "Fast and reliable formal verification of smart contracts with the move prover," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)*, ser. Lecture Notes in Computer Science, vol. 13243. Springer, 2022, pp. 183–200.

[2] J. Park, T. Zhang, W. Grieskamp, M. Xu, G. D. Giacomo, K. Chen, Y. Lu, and R. Chen, "Securing Aptos Framework with Formal Verification," in *5th International Workshop on Formal Methods for Blockchains (FMBC 2024)*, ser. Open Access Series in Informatics (OASIcs), vol. 118. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 9:1–9:16.

[3] W. Grieskamp, "Move goes higher order," https://medium.com/aptoslabs/move-goes-higher-order-cdf9688a4919, Oct. 2025, aptos Labs blog.

[4] K. R. M. Leino and C. Pit-Claudel, "Trigger Selection Strategies to Stabilize Program Verifiers," in *Proceedings of the 28th International Conference on Computer Aided Verification, Part I*. Springer, 2016, pp. 361–381.

[5] C. Barrett, A. Stump, and C. Tinelli, "The SMT-LIB Standard: Version 2.0," in *Proceedings of the 8th International Workshop on Satisfiability Modulo Theories (Edinburgh, UK)*, A. Gupta and D. Kroening, Eds., 2010.

[6] L. M. de Moura and N. Bjørner, "Z3: an efficient SMT solver," in *TACAS*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.

[7] The Move Team, "The Move Programming Language," 2020. [Online]. Available: https://diem.github.io/move

[8] S. Blackshear, D. L. Dill, S. Qadeer, C. W. Barrett, J. C. Mitchell, O. Padon, and Y. Zohar, "Resources: A safe language abstraction for money," 2020. [Online]. Available: https://arxiv.org/abs/2004.05106

[9] K. R. M. Leino, "This is boogie 2," 2008, manuscript KRML 178. [Online]. Available: https://www.microsoft.com/en-us/research/publication/this-is-boogie-2-2/

[10] H. Adams, N. Zinsmeister, and D. Robinson, "Uniswap v2 core," https://uniswap.org/whitepaper.pdf, Mar. 2020, accessed: 2026-04-21.

[11] E. W. Dijkstra, "Guarded commands, nondeterminacy and formal derivation of programs," *Communications of the ACM*, vol. 18, no. 8, pp. 453–457, 1975.

[12] C. B. Jones, *Systematic Software Development Using VDM*, 2nd ed. Prentice-Hall, 1990.

[13] J. M. Spivey, *The Z Notation: A Reference Manual*, 2nd ed. Prentice-Hall, 1992.

[14] W. Grieskamp, "A computational model for Z based on concurrent constraint resolution," in *ZB 2000: Formal Specification and Development in Z and B*, ser. Lecture Notes in Computer Science, J. P. Bowen, S. Dunne, A. Galloway, and S. King, Eds., vol. 1878. Springer, 2000, pp. 414–432.

[15] J.-R. Abrial, *The B-Book: Assigning Programs to Meanings*. Cambridge University Press, 1996.

[16] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel, "Verus: Verifying rust programs using linear ghost types," in *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, 2023, pp. 286–315.

[17] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin, "Dependent types and multi-monadic effects in F*," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2016, pp. 256–270.

[18] K. Maillard, D. Ahman, R. Atkey, G. Martínez, C. Hritcu, E. Rivas, and E. Tanter, "Dijkstra monads for all," in *24th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2019. [Online]. Available: https://arxiv.org/abs/1903.01237

[19] K. M. Leino, "Accessible software verification with dafny," *IEEE Software*, vol. 34, no. 06, pp. 94–97, nov 2017.

[20] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, ser. Lecture Notes in Computer Science, A. Platzer and G. Sutcliffe, Eds., vol. 12699. Springer, 2021, pp. 625–635.

[21] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, ser. Lecture Notes in Computer Science. Springer, 2002, vol. 2283.

[22] Y. Bertot and P. Castéran, *Interactive Theorem Proving and Program Development. Coq'Art: The Calculus of Inductive Constructions*, ser. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004.

[23] Certora, "The Certora Prover," https://www.certora.com/, 2024.

[24] L. Alt and C. Reitwießner, "SMT-Based Verification of Solidity Smart Contracts," in *ISoLA (4)*, ser. Lecture Notes in Computer Science, vol. 11247. Springer, 2018, pp. 376–388.

[25] Á. Hajdu and D. Jovanovic, "solc-verify: A modular verifier for solidity smart contracts," *CoRR*, vol. abs/1907.04262, 2019.

[26] ——, "SMT-Friendly Formalization of the Solidity Memory Model," in *ESOP*, ser. Lecture Notes in Computer Science, vol. 12075. Springer, 2020, pp. 224–250.

Appendix
AI-Based Development Approach

The implementation described in this paper was developed in close collaboration with AI coding assistants — primarily Claude Code, with Codex consulted as a second opinion on critical design decisions. The approach relied on a self-validating feedback loop: verification and specification inference (Sec. IV) were developed in parallel, so that any change to the encoding immediately had a baseline of test cases against which the AI could check itself — the inferred specs must verify end-to-end, with no human in the loop required to assess correctness.

What went well.

- *Specification language extensions in the frontend.* Parser and type-checker changes for behavioral predicates and state labels were largely produced by the AI, with human effort limited to review.
- *Bug identification and fixing.* Once a problematic example was unearthed — typically by inference producing a spec that failed verification — the AI was effective at diagnosing the root cause and proposing a principled fix.
- AI was very good at diagnosing verification problems like timeouts, by analyzing generated Boogie and SMT code, using Z3's logging features to observe quantifier instantiation, etc.
- *Boilerplate and well-understood areas.* Routine code such as expression simplification, AST traversals, and bytecode-level rewrites was handled essentially autonomously.

What went less well.

- *Hand-holding in the complex parts.* The Boogie encoding of behavioral predicates, the discrimination over closure variants, and the existential treatment of intermediate state labels all required substantial human guidance. The AI's first attempts were rarely correct for the general case, and judgement calls about soundness needed to be made by the human.

- *Many iterations.* Reaching a correct design often required dozens of iterations and significant elapsed time; in the most complex areas, the total effort approached that of manual implementation.
- *Token cost.* Long-running iterative work over a large codebase accumulates substantial inference cost.

Overall, the experience was positive. The validation pipeline gave the AI an unambiguous correctness signal independent of human review, and the AI carried a disproportionate share of the work in areas where the design space was already well understood. The harder, design-novel parts remained human-driven — but the AI made the surrounding work substantially less laborious.